1. RSpec calls our around hook, passing it the example we're running.
2. Inside the hook, we tell Sequel to start a new database transaction.
3. Sequel calls the inner block, in which we tell RSpec to run the example.
4. The body of the example finishes running.
5. Sequel rolls back the transaction, wiping out any changes we made to the database.
6. The around hook finishes, and RSpec moves on to the next example.

We only want to spend time setting up the database when we actually use it. Although starting and rolling back a transaction takes less than a tenth of a second, that dwarfs the runtime of our fast, focused unit specs.

This is where the notion of *tagging* comes in. A tag is a piece of metadata—custom information—attached to an example or group. Here, you'll use a new symbol, :db, to indicate that an example touches the database:

06-integration-specs/07/expense_tracker/spec/integration/an_integration_spec.rb

```
require_relative '../support/db'

RSpec.describe 'An integration spec', :db do
  # ...
end
```

Notice that we've had to do two things to make sure this spec runs inside a database transaction:

- Explicitly load the setup code from support/db
- Tag the example group with :db

If you have several spec files that touch the database, it's easy to forget to do one or the other of these things. The result can be specs that pass or fail inconsistently (depending on when and how you run them).

It'd be nice to be able to tag the database-related example groups with :db and trust that the support code will be loaded as needed. Luckily, RSpec has an option that supports exactly this usage. In your spec_helper.rb, add the following snippet inside the RSpec.configure block:

06-integration-specs/07/expense_tracker/spec/spec_helper.rb

```
RSpec.configure do |config|
  config.when_first_matching_example_defined(:db) do
    require_relative 'support/db'
  end
```